```c
#include <stdio.h>

int main() {
    int arr[50] = {1, 2, 3, 4};
    int n = 4;
    int position, element;
    int *ptr = arr;

    printf("Position for insertion: ");
    scanf("%d", &position);

    printf("Element for insertion: ");
    scanf("%d", &element);

    // Shift elements to the right
    for (int i = n; i >= position; i--) {
        *(ptr + i) = *(ptr + i - 1);
    }

    // Insert element
    *(ptr + position - 1) = element;
    n++;

    printf("Array after insertion:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", *(ptr + i));
    }

    return 0;
}
```

```c
#include <stdio.h>

int main() {
    int arr[50] = {1, 2, 3, 4, 5};
    int n = 5;
    int position;
    int *ptr = arr;

    printf("Enter position to delete: ");
    scanf("%d", &position);

    // Check valid position
    if (position < 1 || position > n) {
        printf("Invalid position!");
        return 0;
    }

    // Shift elements to the left
    for (int i = position - 1; i < n - 1; i++) {
        *(ptr + i) = *(ptr + i + 1);
    }

    n--;

    printf("Array after deletion:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", *(ptr + i));
    }

    return 0;
}
```

```c
#include <stdio.h>
#define MAX 20

int main() {
   int poly1[MAX] = {0};
   int poly2[MAX] = {0};
   int diff[MAX] = {0};
   int result[2*MAX] = {0};

   // P1 = 5x4 + 3x2 + 15
   poly1[4] = 5;
   poly1[2] = 3;
   poly1[0] = 15;

   // P2 = 9x8 + 3x7 - x2 - 1
   poly2[8] = 9;
   poly2[7] = 3;
   poly2[2] = -1;
   poly2[0] = -1;

   for(int i = 0; i < MAX; i++) {
      diff[i] = poly1[i] - poly2[i];
   }

   printf("Subtraction Result:\n");
   for(int i = MAX-1; i >= 0; i--) {
      if(diff[i] != 0)
         printf("%dx^%d + ", diff[i], i);
   }
    for(int i = 0; i < MAX; i++) {
      for(int j = 0; j < MAX; j++) {
         result[i+j] += poly1[i] * poly2[j];
      }
   }
   printf("\n");

   printf("Multiplication Result:\n");
   for(int i = 2*MAX-1; i >= 0; i--) {
      if(result[i] != 0)
         printf("%dx^%d + ", result[i], i);
   }

   return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int coeff, power;
   struct Node* next;
};

// Create node
struct Node* create(int c, int p) {
   struct Node* n = (struct Node*)malloc(sizeof(struct Node));
   n->coeff = c;
   n->power = p;
   n->next = NULL;
   return n;
}

// Insert
void insert(struct Node** head, int c, int p) {
   struct Node* temp = create(c,p);
   if(*head == NULL) {
      *head = temp;
      return;
   }
   struct Node* t = *head;
   while(t->next) t = t->next;
   t->next = temp;
}

// Display polynomial
void display(struct Node* head) {
   while(head) {
      printf("%dx^%d", head->coeff, head->power);
      if(head->next && head->next->coeff >= 0)
         printf(" + ");
      else if(head->next)
         printf(" ");
      head = head->next;
   }
   printf("\n");
}

// Addition
struct Node* add(struct Node* p1, struct Node* p2) {
   struct Node* result = NULL;

   while(p1 && p2) {
```

```
        if(p1->power == p2->power) {
            insert(&result, p1->coeff + p2->coeff, p1->power);
            p1 = p1->next;
            p2 = p2->next;
        }
        else if(p1->power > p2->power) {
            insert(&result, p1->coeff, p1->power);
            p1 = p1->next;
        }
        else {
            insert(&result, p2->coeff, p2->power);
            p2 = p2->next;
        }
    }

    while(p1) { insert(&result, p1->coeff, p1->power); p1 = p1->next; }
    while(p2) { insert(&result, p2->coeff, p2->power); p2 = p2->next; }

    return result;
}

// Multiplication
struct Node* multiply(struct Node* p1, struct Node* p2) {
    struct Node* result = NULL;

    for(struct Node* i = p1; i != NULL; i = i->next) {
        for(struct Node* j = p2; j != NULL; j = j->next) {

            int coeff = i->coeff * j->coeff;
            int power = i->power + j->power;

            struct Node* temp = result;
            struct Node* prev = NULL;

            while(temp && temp->power != power) {
                prev = temp;
                temp = temp->next;
            }

            if(temp) {
                temp->coeff += coeff;
            } else {
                struct Node* newNode = create(coeff, power);
                if(prev == NULL) {
                    newNode->next = result;
                    result = newNode;
                } else {
```

```
                newNode->next = prev->next;
                prev->next = newNode;
            }
        }
    }
    return result;
}

int main() {
    struct Node *P1 = NULL, *P2 = NULL;

    // P1 = 5x4 + 3x2 + 15
    insert(&P1, 5, 4);
    insert(&P1, 3, 2);
    insert(&P1, 15, 0);

    // P2 = 9x8 + 3x7 - x2 - 1
    insert(&P2, 9, 8);
    insert(&P2, 3, 7);
    insert(&P2, -1, 2);
    insert(&P2, -1, 0);

    printf("Polynomial 1:\n");
    display(P1);

    printf("Polynomial 2:\n");
    display(P2);

    struct Node* sum = add(P1, P2);
    printf("\nAddition Result:\n");
    display(sum);

    struct Node* prod = multiply(P1, P2);
    printf("\nMultiplication Result:\n");
    display(prod);

    return 0;
}
```

```c
#include <stdio.h>
#define MAX 50

int stack[MAX];
int top = -1;

// Push function
void push(int value) {
   if (top == MAX - 1) {
      printf("Stack Overflow\n");
      return;
   }
   top++;
   stack[top] = value;
}

// Pop function
int pop() {
   if (top == -1) {
      printf("Stack Underflow\n");
      return -1;
   }
   return stack[top--];
}

int main() {
   int arr[50];
   int n;
   printf("enter number of elements:");
   scanf("%d",&n);
   printf("enter elements:");
   for(int i = 0; i < n; i++){
      scanf("%d",&arr[i]);
   };

   printf("Array: ");
   for(int i = 0; i < n; i++) {
      printf("%d ", arr[i]);
   }
```

```
    for(int i = 0; i < n; i++) {
        push(arr[i]);
    }

    for(int i = 0; i < n; i++) {
        arr[i] = pop();
    }

    printf("\nReversed Array: ");
    for(int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

```
enter number of elements:6
enter elements:2
3
4
5
6
7
Array: 2 3 4 5 6 7
Reversed Array: 7 6 5 4 3 2
```